

โครงงานการเขียนโปรแกรมเชิงวัตถุด้วยภาษา Java
Strang Sushi Game

โดย

นางสาวโชติกา เทพษร

รหัสนิสิต 68160002

เสนอ

ดร. คนึงนิจ กุโบลา

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา หลักการโปรแกรมเชิงวัตถุ
สาขาวิชา เทคโนโลยีสารสนเทศเพื่ออุตสาหกรรมดิจิทัล
มหาวิทยาลัยบูรพา วิทยาเขตบางแสน
ภาคเรียนที่ 2 ปีการศึกษา 2568

คำนำ

รายงานเล่มนี้จัดทำขึ้นเพื่อเป็นส่วนหนึ่งของรายวิชา หลักการโปรแกรมเชิงวัตถุ (Object-Oriented Programming) เพื่อศึกษาและทำความเข้าใจเกี่ยวกับการพัฒนาระบบด้วยภาษา Java อย่างลึกซึ้ง ตลอดจนได้ลงมือปฏิบัติจริงเพื่อให้เกิดทักษะที่เป็นประโยชน์ต่อการเรียนและการทำงานในอนาคต

ผู้จัดทำหวังเป็นอย่างยิ่งว่า รายงานเล่มนี้จะเป็นประโยชน์ต่อผู้อ่าน นักเรียน หรือนักศึกษา ที่กำลังค้นคว้าข้อมูลเกี่ยวกับการเขียนโปรแกรมเชิงวัตถุ หากมีข้อบกพร่องหรือข้อผิดพลาดประการใด ผู้จัดทำขอน้อมรับคำแนะนำและขออภัยมา ณ ที่นี้

ผู้จัดทำ
วันที่ 26 มี.ค. 69

สารบัญ

คำนำ 2

สารบัญ 3

บทที่ 1 บทนำ 5

1.1 ชื่อโครงงาน 5

1.2 ประเภทของโครงงาน 5

1.3 แนวคิดและแรงบันดาลใจ 5

1.4 วัตถุประสงค์ของโครงงาน 5

1.5 ขอบเขตของโครงงาน 5

บทที่ 2 การวิเคราะห์ระบบ 7

2.1 วิเคราะห์ความต้องการ 7

2.1.1 Functional Requirements 7

2.1.2 Non-Functional Requirements 7

บทที่ 3 การออกแบบระบบ 8

3.1 การออกแบบโครงสร้างโปรแกรมตามหลัก OOP 8

3.2 การออกแบบหน้าจอ (UI Design) 9

3.3 การออกแบบขั้นตอนการทำงาน 10

บทที่ 4 การพัฒนาโปรแกรม 11

4.1 เครื่องมือที่ใช้ 11

4.2 โครงสร้างคลาสของระบบ (Class Structure) 11

บทที่ 5 การทดสอบโปรแกรม 13

5.1 วิธีการทดสอบ 13

5.2 ตารางผลการทดสอบ 14

บทที่ 6 ปัญหาและอุปสรรค 15

6.1 ปัญหาที่พบ 15

6.2 วิธีแก้ไข 15

6.3 สิ่งที่ได้เรียนรู้จากการทำโครงงาน 15

บทที่ 7 สรุปผลและข้อเสนอแนะ 16

7.1 โครงงานบรรลุวัตถุประสงค์หรือไม่ 16
7.2 หลัก OOP ช่วยให้โปรแกรมมีโครงสร้างที่ดีขึ้นอย่างไร 16
7.3 แนวทางพัฒนาเพิ่มเติมในอนาคต 16
บทที่ 8 ภาคผนวก 17
8.1 Source Code 17
8.1.1 Class Main : 17
8.1.2 Class GamePannel : 18
8.1.3 Class GameManager : 25
8.1.4 Class GameChar : 26
8.1.5 Class Customer : 26
8.1.6. Class Ghost : 28
8.1.7 Class Menu : 29
8.1.8 Class Recipe : 29
8.1.9 Class SoundManager : 30
8.2 คู่มือการใช้งาน 32
8.3 วิธีรันโปรแกรม 32
8.4 UML Diagram 33

บทที่ 1 บทนำ

1.1 ชื่อโครงงาน

- Strange Sushi Game

1.2 ประเภทของโครงงาน

- เกม (Game Application)

1.3 แนวคิดและแรงบันดาลใจ

ด้วยแรงบันดาลใจจากเกมทำซูชิที่เคยเล่น ประกอบกับความสนใจในการพัฒนาเกมร้านอาหาร ผนวกกับแรงบันดาลใจจากเกมแนวระทึกขวัญสยองขวัญ ‘No, I’m not human’ ที่มีการปรับเปลี่ยนรูปลักษณ์ตัวละครให้ผิดแผกไปจากมนุษย์ทั่วไปแต่ยังคงความสมจริง ทำให้ผู้เล่นต้องสังเกตและระมัดระวังว่าบุคคลที่พบเห็นเป็นมนุษย์จริงหรือไม่ จากแรงบันดาลใจของทั้งสองเกม จึงนำมาสู่การพัฒนาเกม Strange Sushi Game ที่มีการดำเนินเกมคล้ายคลึงกับเกมทำอาหารทั่วไป แต่มีบรรยากาศโดยรวมและรูปลักษณ์ของตัวละครที่ได้รับอิทธิพลมาจากเกม ‘No, I’m not human’ โดยผู้เล่นจะได้รับบทเป็นเจ้าของร้านอาหารในสถานที่ซึ่งมีลูกค้าที่มีรูปร่างหน้าตาประหลาด และอาจเผชิญกับปรากฏการณ์เหนือธรรมชาติ ข้อควรระวังในการเล่นคือ ห้ามทำอาหารผิดพลาด และหลีกเลี่ยงการทำให้ลูกค้าไม่พอใจ

1.4 วัตถุประสงค์ของโครงงาน

1. เพื่อฝึกใช้หลักการ OOP ในการพัฒนาโปรแกรม
2. เพื่อออกแบบโครงสร้างโปรแกรมอย่างเป็นระบบ
3. เพื่อประยุกต์ความรู้ภาษา Java
4. เพื่อได้ทดลองสร้างโปรแกรมเป็นระบบจากความรู้ที่ได้ศึกษามา

1.5 ขอบเขตของโครงงาน

Strange Sushi Game เป็นเกมที่ผู้เล่นสวมบทบาทเป็นเจ้าของร้านซูชิ ดำเนินกิจการขายซูชิให้แก่ลูกค้าในแต่ละวัน โดยมีระบบและองค์ประกอบหลักดังนี้:

- ระบบวัน: ผู้เล่นต้องจำหน่ายซูชิให้สำเร็จในแต่ละวัน
- ค่าเช่ารายวัน: ผู้เล่นจะต้องชำระค่าเช่ารายวันที่เพิ่มขึ้นในแต่ละวัน
- ระบบเมนู: ผู้เล่นต้องเลือกวัตถุดิบเพื่อปรุงอาหารแต่ละเมนูตามความต้องการของลูกค้า หากเลือกวัตถุดิบผิดพลาดหรือไม่ครบถ้วน ลูกค้าจะไม่พอใจและจะโดนหักเงิน
- Jumpscare: หากผู้เล่นทำอาหารผิดพลาดหรือทำให้ลูกค้าไม่พอใจบ่อยครั้งก็จะมีโอกาสที่จะถูก Jumpscare และหากผู้เล่นทำอาหารให้ลูกค้าผีผิดจะโดน Jump Scare ทันที
- ระบบลงโทษ: บทลงโทษจะรุนแรงขึ้น โดยจะหักเงินมากขึ้นตามจำนวนวันที่ผ่านไปเมื่อผู้เล่นทำอาหารผิดพลาด

- ระบบดูเมนู: หากผู้เล่นลืมขั้นตอนการทำอาหาร สามารถตรวจสอบสูตรอาหารได้ที่ปุ่ม "Recipe"
- ระบบยกเลิกการเลือก: หากผู้เล่นเลือกวัตถุดิบผิดพลาด สามารถยกเลิกการเลือกได้ที่ปุ่ม "Clear"
- ระบบสิ้นสุดเกม: เมื่อผู้เล่นมียอดเงินรวมต่ำกว่าศูนย์ หรือไม่สามารถชำระค่าเช่าตามเงื่อนไขที่กำหนด เกมจะสิ้นสุดลงทันที
- ระบบแสดงข้อมูล: อินเทอร์เฟซของเกมจะปรากฏข้อมูลวัน เวลา และยอดเงินคงเหลือของผู้เล่นในขณะนั้น
- ระบบสุ่มเมนูอาหาร: เกมมีฟังก์ชันสุ่มเมนูอาหารจำนวน 6 รายการ เพื่อเพิ่มความหลากหลายในกระบวนการปรุงอาหารให้กับผู้เล่นเกม

Strange Sushi Game เป็นเกมที่ไม่มีระบบบันทึกความคืบหน้า หากผู้เล่นออกจากเกมกลางคัน ข้อมูลการเล่นจะไม่ถูกบันทึก ทำให้เกมนี้เป็นเกมในระบบ No Save Points

สิ่งที่ไม่อยู่ในขอบเขตของโครงงาน

- ระบบอัปเกรดร้านค้า: ณ ตอนนี้ยังไม่มีระบบอัปเกรดร้านค้า หรือ วัตถุดิบ
- ระบบบันทึกเกม: ไม่มีระบบบันทึกข้อมูล เพื่อที่จะสามารถกลับมาเล่นต่อได้
- ระบบตกแต่งร้านค้า: ไม่มีระบบที่ผู้เล่นจะสามารถตกแต่งร้านค้าได้ตามต้องการ
- ระบบสื่อสารกับลูกค้า: ไม่มีระบบที่สามารถตอบโต้ หรือ สื่อสารกับลูกค้าได้
- ระบบจำนวนวัตถุดิบ: ไม่มีระบบจำนวนวัตถุดิบที่สามารถใช้ได้ในแต่ละครั้ง
- ระบบซื้อวัตถุดิบ: ไม่มีระบบที่ผู้เล่นสามารถซื้อวัตถุดิบได้ เมื่อวัตถุดิบหมด

บทที่ 2 การวิเคราะห์ระบบ

2.1 วิเคราะห์ความต้องการ

2.1.1 Functional Requirements

- ระบบสามารถเริ่มต้นการทำงานและผู้เล่นเข้าสู่หน้าจอหลักของเกมได้
- ระบบสามารถประมวลผลข้อมูลและตรรกะของเกมได้อย่างถูกต้อง
- ระบบสามารถตรวจสอบเงื่อนไขและแสดงผลลัพธ์ได้อย่างถูกต้องตามที่กำหนด
- ส่วนควบคุมการทำงานสามารถรับคำสั่งและทำงานได้อย่างถูกต้อง
- ระบบสามารถคำนวณยอดเงินคงเหลือและเวลาภายในเกมได้อย่างถูกต้องแม่นยำ
- ระบบสามารถสุ่มเมนูอาหารได้ประเภทต่างๆได้ตามเงื่อนไขที่กำหนด

2.1.2 Non-Functional Requirements

- ระบบมีความเสถียร โปรแกรมไม่หยุดทำงานระหว่างการใช้งาน
- ส่วนติดต่อผู้ใช้งาน ออกแบบให้เข้าใจง่ายและผู้เล่นสามารถทำความเข้าใจวิธีการเล่นได้
- โครงสร้าง ได้รับการออกแบบและพัฒนาตามหลักการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming) อย่างถูกต้อง
- ระบบตอบสนองต่อคำสั่งของผู้เล่นได้ เพื่อให้เกมมีความลื่นไหลและไม่สะดุด

บทที่ 3 การออกแบบระบบ

3.1 การออกแบบโครงสร้างโปรแกรมตามหลัก OOP

1. การห่อหุ้มข้อมูล (Encapsulation)

ตัวอย่างเช่น การใช้ method ประเภท Getter อย่าง getCoin() เพื่ออ่านค่าจำนวนเงิน และใช้ method takeCoin() เพื่อปรับลดจำนวนเงินตามเงื่อนไขตรรกะของเกมที่ได้กำหนดไว้

```
public class GameManager {
      private double coin;
            public double getCoin() {  return coin;  } //getter
            public void takeCoin(double coin) { this.coin -= coin;
```

2. ความสัมพันธ์ระหว่างคลาส (Association)

ตัวอย่างเช่น คลาส GamePannel มีการเรียกใช้ object จากคลาส GameManager และคลาส Recipe เข้ามาเป็นส่วนหนึ่งของการประมวลผลการทำงาน

```
public class GamePannel extends JPanel {
      private final GameManager gameManager;
      private final Recipe recipe;
```

3. การสืบทอดคุณสมบัติ (Inheritance)

ตัวอย่างเช่น คลาส Ghost ทำการสืบทอดคุณสมบัติมาจากคลาส Customer และมีการใช้คำสั่ง super() เพื่อเรียกใช้งาน Constructor ของคลาสหลัก

```
public class Ghost extends Customer {
    private int ghostType;
        public Ghost(int ghostType, GameManager gameManager) {
        super();
```

4. การพ้องรูป (Polymorphism)

ตัวอย่างเช่น การกำหนดตัวแปร currentCustomer หากสร้าง object ด้วย new Customer() จะเป็นลูกค้าปกติ แต่หากกำหนด object เป็น ghost เมื่อมีการเรียกใช้คำสั่งระบบจะแสดงผลพฤติกรรมที่แตกต่างออกไป

```
Customer currentCustomer;
currentCustomer = new Customer();
currentCustomer = ghost;
currentCustomer.react(gameManager);
```

5. การเขียนทับอด (Method Overriding)

ตัวอย่างเช่น คลาส Ghost ที่สืบทอดมาจากคลาส Customer ได้ทำการเขียนทับ method เดิม เพื่อปรับเปลี่ยนพฤติกรรมและการโต้ตอบให้สอดคล้องกับลักษณะเฉพาะของลูกค้าที่เป็นผี

```
@Override
public void react(GameManager gm) { ... }
@Override
public void onLeave(GameManager gm) { ... }
```

6. การโอเวอร์โหลดเมธอด (Method Overloading)

ตัวอย่างเช่น ในคลาส Customer มีการสร้าง method ตอบสนองสองรูปแบบ รูปแบบแรกใช้สำหรับพฤติกรรมปกติ และรูปแบบที่สองมีการรับ parameter เพิ่มเติม เพื่อนำมาประมวลผลบทลงโทษที่รุนแรงขึ้น

```
public void react(GameManager gm) { ... }
public void react(GameManager gm, int penalty) { ... }
```

7. คลาสนามธรรม (Abstract Class)

ตัวอย่างเช่น abstract class GameChar เพื่อกำหนดโครงสร้างหลัก และบังคับให้คลาสลูกที่สืบทอดไปต้องจัดการรายละเอียดของ method ตามที่บังคับไว้ ซึ่งคลาสลูกในที่นี้คือ Customer และ Ghost

```
public abstract class GameChar {
      public abstract void react(GameManager gm);
      public abstract void onLeave(GameManager gm);
}
```

3.2การออกแบบหน้าจอ (UI Design)

หน้าเมนูหลัก (Main Menu)

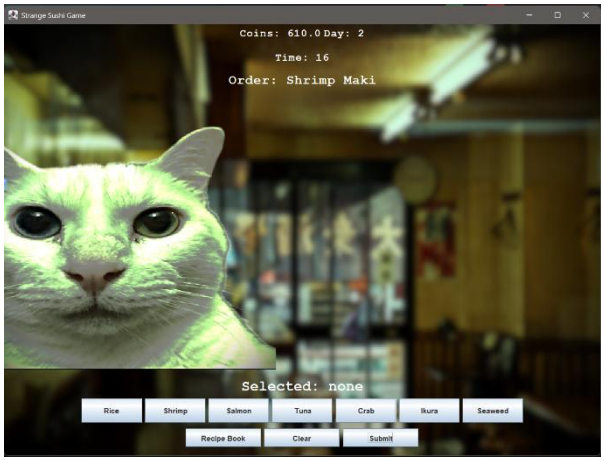
แสดงชื่อเกมและปุ่มคำสั่งหลัก ได้แก่ ปุ่ม “Start” สำหรับเข้าสู่เกมเพื่อเริ่มต้นเล่น และปุ่ม “How to Play” สำหรับแสดงคู่มือวิธีการเล่นพื้นฐาน เพื่อให้ผู้เล่นทำความเข้าใจระบบและเตรียมความพร้อมก่อนเริ่มเล่นจริง

หน้าจอการเล่น (Gameplay Screen)

แสดงข้อมูลและสถานะที่จำเป็นสำหรับการเล่น เช่น ลำดับวัน จำนวนเงิน เวลาที่เหลือ รายการอาหารที่ลูกค้าสั่ง ส่วนควบคุมการทำงาน และตัวละครลูกค้า โดยออกแบบส่วนติดต่อผู้ใช้งานให้เข้าใจง่าย



Main Menu



Gameplay Screen

3.3 การออกแบบขั้นตอนการทำงาน

1. เริ่มต้นการทำงานของโปรแกรม
2. โหลดหน้าต่างเกม (Main menu)
3. ผู้เล่นกดปุ่ม “Start” เพื่อทำการเริ่มตัวเกม
4. โหลดหน้าต่างการเล่น (Gameplay Screen)
5. ระบบจะทำการสุ่มสร้างตัวละครลูกค้าซึ่งอาจเป็นได้ทั้ง ลูกค้าปกติ หรือ ผี
6. ระบบแสดงออเดอร์เมนูอาหารที่ลูกค้าต้องการ พร้อมทั้งเริ่มนับถอยหลังเวลาการรอ
7. ระบบรับคำสั่งจากผู้เล่นในการเลือกวัตถุดิบและกด “Submit”
8. **ตรวจสอบเงื่อนไขการเสิร์ฟอาหาร** ตรวจสอบว่าผู้เล่นทำอาหารได้ถูกต้องตามออเดอร์ และเสิร์ฟทันเวลาหรือไม่

   8.1 **กรณีที่ถูกต้องและทันเวลา** ระบบจะทำการเพิ่มจำนวนเงินให้ผู้เล่นตามมูลค่าของอาหาร และลูกค้าใหม่จะถูกสร้างขึ้นมา

   8.2 **กรณีที่ไม่ถูกต้องหรือหมดเวลารอก่อน** ระบบจะเรียกใช้ method การตอบสนอง ซึ่งจะทำการหักเงินผู้เล่น และแสดงข้อความเพื่อแสดงว่าลูกค้าโกรธ หรือ Jumpscare ในกรณีที่เป็นผี จากนั้นลูกค้าใหม่จะถูกสร้างขึ้นมา
9. **ตรวจสอบเงื่อนไขการจบเกม** ระบบจะตรวจสอบว่าเงินของผู้เล่นติดลบ หรือหมดเวลาของด่านนั้นแล้วหรือไม่

   9.1 **กรณีที่ยังไม่จบเกม** ระบบจะวนกลับไปทำงานที่ **ขั้นตอนที่ 3** เพื่อสุ่มลูกค้าคนต่อไปเข้ามาในร้าน

   9.2 **กรณีที่ตรงกับเงื่อนไขจบเกม** ระบบจะหยุดการสุ่มลูกค้า และเปลี่ยนไปแสดงข้อความจบเกม
10. **สิ้นสุดการทำงาน**

บทที่ 4 การพัฒนาโปรแกรม

4.1 เครื่องมือที่ใช้

- **ภาษา Java :** Java (JDK 17)
- **IDE ที่ใช้ :** IntelliJ IDEA
- **Library หรือ Framework ที่ใช้ :**
  - Java Swing (javax.swing.*) เป็น Library มาตรฐานของ Java
  - Java AWT (java.awt.*) เพื่อจัดการเรื่องกราฟิก
  - Java Sound (javax.sound.sampled.*) เป็น Library สำหรับจัดการระบบเสียง

4.2 โครงสร้างคลาสของระบบ (Class Structure)

1. คลาสหลักของระบบ (Main Classes)

- **คลาส Main** เป็นคลาสเริ่มต้นของโปรแกรม ทำหน้าที่เรียกใช้งานหน้าต่างเกมและเริ่มต้นระบบ
- **คลาส GameManager** เป็นคลาสที่ใช้ควบคุมตรรกะของเกม เช่น การจัดการเงิน, และการควบคุมเวลาของระบบ
- **คลาส GamePannel** เป็นคลาสที่จัดการด้านการแสดงผลกราฟิก (GUI) รูปภาพตัวละคร การนำเข้าฟอนต์อักษรพิเศษ และการรับคำสั่งจากผู้เล่น

2. คลาสแม่ / คลาสลูก

- **คลาสแม่** คลาส Customer ทำหน้าที่เก็บคุณสมบัติพื้นฐานที่ลูกค้าทุกคนต้องมี เช่น ออเดอร์อาหาร, เวลารอคอย, และสถานะความโกรธ
- **คลาสลูก** คลาส Ghost มีการสืบทอดคุณสมบัติมาจากคลาส Customer ทำให้ตัวละครผีมีระบบการสั่งอาหารและเวลารอคอยเหมือนลูกค้าปกติ แต่เพิ่มคุณสมบัติเฉพาะตัวเข้าไปได้ ก็คือการแสดงภาพ Jumpscare

3. Abstract Class

- **คลาส GameChar** (ตัวละครในเกม) เป็น Abstract Class เพื่อเป็นต้นแบบของตัวละครทั้งหมด โดยมีการกำหนด Abstract Method ชื่อว่า react(); และ onLeave(); เป็นโครงร่าง เพื่อบังคับให้คลาสลูก ต้องนำ method นี้ไปเขียนการทำงาน

4. **Overloading และ Overriding**

- **Overriding** คลาส Ghost ที่สืบทอดมาจากคลาส Customer ได้ทำการเขียนทับ method เดิม เพื่อปรับเปลี่ยนพฤติกรรมและการโต้ตอบให้สอดคล้องกับลักษณะเฉพาะของลูกค้าที่เป็นผี

```
@Override
public void react(GameManager gm) { ... }
@Override
public void onLeave(GameManager gm) { ... }
```

- **Overloading** ในคลาส Customer มีการสร้าง method ตอบสนองสองรูปแบบ รูปแบบแรกใช้สำหรับพฤติกรรมปกติ และรูปแบบที่สองมีการรับ parameter เพิ่มเติม เพื่อนำมาประมวลผลบทลงโทษที่รุนแรงขึ้น

```
public void react(GameManager gm) { ... }
public void react(GameManager gm, int penalty) { ... }
```

บทที่ 5 การทดสอบโปรแกรม

5.1 วิธีการทดสอบ

ทดสอบฟังก์ชันหลักของระบบ

- ทดสอบการคำนวณ คิดเลข จ่ายค่าเช่า เงินคงเหลือถูกต้อง
- ทดสอบการสุ่มของเมนูแต่ละเมนู
- ทดสอบระบบใส่วัตถุดิบซ้ำหรือวัตถุดิบผิดควรที่จะหักเงินผู้เล่น
- ทดสอบระบบ Jumpscare ไม่ให้เกิดการทำซ้ำ
- ทดสอบระบบจบเกม เมื่อเงินติดลบหรือเป็นศูนย์
- ทดสอบระบบจับเวลา
- ทดสอบการกดปุ่มต่าง ๆ ให้ทำงานได้เหมาะสม

ทดสอบการทำงานของคลาสต่าง ๆ

- ทดสอบคลาส Ghost และ Customer มีการแสดงรูปภาพที่ตรงต่อประเภทของลูกค้า
- ทดสอบคลาส SoundManager ควรทำงานตามที่กำหนด มีการหยุดชะงักหรือไม่
- ทดสอบคลาส GameManager มีการจบวันตามที่กำหนดหรือไม่
- ทดสอบคลาส GamePannel มีการอัปเดตข้อมูลเป็นข้อมูล ณ ปัจจุบันหรือไม่

5.2 ตารางผลการทดสอบ

ลำดับ	กรณีทดสอบ	ผลที่คาดหวัง	ผลลัพธ์จริง	ผ่าน/ไม่ผ่าน
1	**การเริ่มต้นเกม**	ตัวเกมสามารถเริ่มต้นได้ ไม่มีปัญหาเกิดขึ้น	เริ่มต้นเกมได้ ไม่เกิดปัญหาใด ๆ	✓
2	**ไม่เกิดการ Jumpscare ซ้ำ**	จะไม่ทำการ Jumpscare เมื่อทำเมนูให้ลูกค้าผีเสร็จและเมื่อเวลาหมด	ไม่เกิดการทำซ้ำ	✓
3	**แสดงข้อมูลการสุ่มเมนู**	ระบบสุ่มเมนูตามที่ได้กำหนดไว้ทุกเมนู	รายชื่อเมนูมีการสุ่มอย่างถูกต้อง	✓
4	**แสดงข้อมูลการสุ่มลูกค้า**	ระบบสุ่มลูกค้าทั้งสองประเภทคือ ลูกค้าปกติ และ ลูกค้าผี	รูปลูกค้ามีการสุ่มตามที่กำหนด	✓
5	**การกดปุ่ม “Recipe”**	มีการแสดงรูปภาพวิธีการทำอาหารแต่ละเมนู	มีการแสดงรูปภาพขึ้น	✓
6	**การจับเวลาทำงานได้**	เวลาลดตามปกติ ไม่ติดขัด	เวลาลดตามปกติ	✓
7	**การจบเกมเมื่อเงินติดลบ**	เกมแสดงข้อความจบเกม	ระบบแสดงข้อความจบเกม	✓
8	**การจบเกมเมื่อเงินเป็นศูนย์**	เกมแสดงข้อความจบเกม	ระบบแสดงข้อความจบเกม	✓
9	**การคิดเงินคงเหลือหลังจากการจ่ายค่าเช่า**	ระบบคิดเลขให้ผู้เล่น โดยที่การคำนวณจะต้องถูกต้องคำหลัก	ระบบคิดเลขได้อย่างถูกต้อง	✓
10	**เมื่อเล่นไปเรื่อย ๆ ราคาแต่ละเมนูจะเพิ่มขึ้น**	ระบบต้องเพิ่มราคาแต่ละเมนูเมื่อเกินวันที่ 3 เพื่อความสมดุล	ราคาเมนูเพิ่มตามที่กำหนดไว้	✓

บทที่ 6 ปัญหาและอุปสรรค

6.1 ปัญหาที่พบ

- เมื่อค่าเงินติดลบ ระบบยังคงสุ่มลูกค้าอย่างต่อเนื่อง ทำให้ผู้เล่นต้องเสิร์ฟอาหารให้ถูกต้องตามกำหนด ระบบจึงจะสรุปผลและแสดงข้อความจบเกม
- หากค่าเงินติดลบหลังจากเสิร์ฟอาหาร ระบบยังคงสุ่มลูกค้าเพิ่มอีก 1 คน และจะแสดงผลจบเกมก็ต่อเมื่อให้บริการลูกค้าคนดังกล่าวเสร็จสิ้น
- เมื่อเริ่มเกม รูปภาพของตัวละครลูกค้าทั้งสองประเภทไม่แสดงผล แต่ระบบยังคงสุ่มออเดอร์และให้ผู้เล่นเสิร์ฟอาหารได้ตามปกติ
- ระบบไม่มีการบังคับหยุดเสียง ทำให้เสียงยังคงเล่นต่อไปแม้เหตุการณ์นั้นจะสิ้นสุดลง ส่งผลให้เกิดการเล่นเสียงซ้อนทับกัน

6.2 วิธีแก้ไข

- เพิ่มการตรวจสอบเงื่อนไขการจบเกมก่อนสุ่มลูกค้าคนต่อไป เพื่อป้องกันการสุ่มลูกค้าใหม่เมื่อค่าเงินเป็นศูนย์หรือติดลบ
- ตรวจสอบที่อยู่ (Path) ของไฟล์รูปภาพ และแก้ไขโดยย้ายโฟลเดอร์รูปภาพมาไว้ในโฟลเดอร์ของโครงงาน (Project Folder) เพื่อให้โปรแกรมเรียกใช้งานได้ถูกต้อง
- เพิ่มการตรวจสอบเงื่อนไขหลังจากการหักเงินผู้เล่น เพื่อป้องกันไม่ให้ระบบสุ่มลูกค้าเพิ่ม และสั่งให้แสดงผลจบเกมทันที
- สร้างเมธอด (Method) สำหรับหยุดเสียงโดยเฉพาะ เพื่อให้สามารถควบคุมการเล่นและการหยุดเสียงได้อย่างมีประสิทธิภาพ

6.3 สิ่งที่ได้เรียนรู้จากการทำโครงงาน

- ได้เรียนรู้วิธีพัฒนาโปรแกรมที่มีความซับซ้อน ซึ่งต้องอาศัยการวางแผนและออกแบบโครงสร้างระบบให้ถูกต้องตามหลักการ เพื่อให้โปรแกรมทำงานได้อย่างสมบูรณ์
- ได้ฝึกทักษะการสืบค้นข้อมูลเพื่อนำมาเพิ่มฟีเจอร์ให้โปรแกรม เช่น การจัดการระบบเสียง (SoundManager) และการสร้างเมธอดนับเวลาถอยหลัง (setTimeLimit)
- ได้ทำความเข้าใจหลักการ OOP และนำมาประยุกต์ใช้เพื่อเขียนโค้ดให้เป็นระเบียบ เข้าใจง่าย และสะดวกต่อการแก้ไขหรือค้นหาการทำงานของระบบ
- ได้เรียนรู้การออกแบบขั้นตอนการทำงานและจัดทำแผนภาพ (Diagram) ซึ่งช่วยให้เห็นภาพรวมของระบบชัดเจนขึ้น และเป็นประโยชน์ต่อการพัฒนาหรือเพิ่มข้อมูลในอนาคต

บทที่ 7 สรุปผลและข้อเสนอแนะ

7.1 โครงงานบรรลุวัตถุประสงค์หรือไม่

โครงงานนี้บรรลุวัตถุประสงค์ตามที่ได้กำหนดไว้ โดยอาศัยหลักการโปรแกรมเชิงวัตถุ (OOP) มาเป็นโครงร่างสำคัญในการวางแผนและพัฒนา ส่งผลให้ระบบสามารถทำงานได้อย่างถูกต้องตามเป้าหมาย พร้อมทั้งมีการเพิ่มฟีเจอร์ที่หลากหลาย ทำให้ตัวเกมมีความน่าสนใจมากยิ่งขึ้น นอกจากนี้ การลงมือปฏิบัติและพัฒนาโครงงานดังกล่าวยังถือเป็นการฝึกฝน และพัฒนาทักษะความรู้ด้านการเขียนโปรแกรมของผู้จัดอีกด้วย

7.2 หลัก OOP ช่วยให้โปรแกรมมีโครงสร้างที่ดีขึ้นอย่างไร

- **การแบ่งการทำงานเป็นสัดส่วน** แยกการทำงานออกเป็นคลาสที่ชัดเจน ช่วยให้โค้ดมีความเป็นระเบียบ ไม่ปะปนกัน ทำให้การค้นหาและแก้ไขข้อผิดพลาด สามารถทำได้ง่ายและตรงจุด
- **การลดการซ้ำของโค้ด** การใช้หลักการสืบทอดคุณสมบัติทำให้คลาสลูกสามารถนำโครงสร้างของคลาสแม่มาใช้งานและพัฒนาต่อได้ทันที ช่วยลดระยะเวลาในการเขียนโค้ดชุดเดิมซ้ำๆ
- **ความปลอดภัยและการบำรุงรักษา** หลักการห่อหุ้มข้อมูล ช่วยควบคุมสิทธิ์การเข้าถึงและป้องกันไม่ให้ตัวแปรสำคัญถูกแก้ไขโดยไม่ตั้งใจ ทำให้ระบบมีความเสถียรและดูแลรักษาได้ง่ายในระยะยาว
- **ความยืดหยุ่นในการรองรับอนาคต** หลักการ Polymorphism และการใช้ Abstraction ทำให้โครงสร้างโปรแกรมมีความยืดหยุ่นสูง หากต้องการเพิ่มฟีเจอร์ ตัวละคร หรือเมนูใหม่ๆ ในอนาคต ก็สามารถทำได้ทันทีโดยไม่ส่งผลกระทบต่อระบบการทำงานเดิม

7.3 แนวทางพัฒนาเพิ่มเติมในอนาคต

แนวทางในการพัฒนาเพิ่มเติม เพื่อให้โปรแกรมน่าสนใจและสนุกมากขึ้น สามารถเพิ่มเติมหัวข้อต่อไปนี้ได้

- **ระบบอัปเกรดร้านค้า** ผู้เล่นสามารถพัฒนาหรืออัปเกรดประสิทธิภาพของอุปกรณ์และวัตถุดิบภายในร้านได้
- **ระบบตกแต่งร้านค้า** ผู้เล่นสามารถปรับแต่งรูปแบบและตกแต่งร้านค้าได้ตามความต้องการ
- **ระบบจัดการวัตถุดิบ** เพิ่มระบบการซื้อวัตถุดิบระหว่างการเล่น หากวัตถุดิบหมด ผู้เล่นจะต้องบริหารเวลาในการสั่งซื้อให้ทันก่อนที่เวลารอของลูกค้าจะหมดลง
- **การปรับปรุงส่วนติดต่อผู้ใช้งาน (User Interface)** ออกแบบหน้าจอการเล่นให้สวยงามและน่าสนใจยิ่งขึ้น เช่น การใช้รูปภาพวัตถุดิบแทนปุ่มกดข้อความ เพื่อเพิ่มอรรถรสในการเล่น
- **ระบบบันทึกข้อมูล**เพื่อให้ผู้เล่นสามารถบันทึกความคืบหน้าของเกมและกลับมาเล่นต่อได้ในภายหลังโดยไม่จำเป็นต้องเริ่มต้นเกมใหม่ทุกครั้ง

บทที่ 8 ภาคผนวก

8.1 Source Code

8.1.1 Class Main :

```
public class Main {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Strange Sushi Game");
        frame.setSize(1000, 750);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        //----------------------- Menu Panel ----------------------------
        JPanel menuPanel = new JPanel(new BorderLayout());
        menuPanel.setBackground(Color.BLACK); // black background

        // title label
        JLabel titleLabel = new JLabel("Strange Sushi Game",
SwingConstants.CENTER);
        titleLabel.setFont(new Font("Courier New", Font.BOLD, 40));
        titleLabel.setForeground(Color.WHITE);

        JButton howtoBtn = new JButton("How to Play");
        howtoBtn.setFont(new Font("Courier New", Font.BOLD, 24));
        howtoBtn.addActionListener(e -> {
            JOptionPane.showMessageDialog(null,"How to play: \n" +
                    " 1. Choose all the ingredient to make Sushi!" + "\n" +
                    " 2. Then click 'Submit' to serve order" + "\n" +
                    " 3. Made mistake? Click 'Clear' to choose ingredient again"
+ "\n" +
                    " 4. Click 'Recipe' to look at the recipe"
                    ,"How to play?",JOptionPane.INFORMATION_MESSAGE);
        });

        // start button
        JButton startBtn = new JButton("Start Game");
        startBtn.setFont(new Font("Courier New", Font.BOLD, 24));

        // center the button
        JPanel btnPanel = new JPanel();
        btnPanel.setBackground(Color.BLACK);
        btnPanel.add(startBtn);
        btnPanel.add(howtoBtn);

        menuPanel.add(titleLabel, BorderLayout.CENTER);
        menuPanel.add(btnPanel, BorderLayout.SOUTH);

        // switch to game on click
        startBtn.addActionListener(e -> {
            frame.getContentPane().removeAll();
            frame.add(new GamePannel(frame));
            frame.revalidate();
            frame.repaint();
        });

        frame.add(menuPanel);
        frame.setVisible(true);
    }
}
```

8.1.2 Class GamePannel :

```
public class GamePannel extends JPanel { // doesn't have to add JPanel
    private final ArrayList<String> playerSelection; // tracks what
player clicked
    private final GameManager gameManager;  // to talk to GameManager
    private Customer currentCustomer; // who is currently at the counter
    private final JLabel customerLabel;     // shows customer image
    private final JLabel coinLabel;         // shows current money
    private final JLabel dayLabel;          // shows current day
    private final Recipe recipe;            // for random dish orders
    private final JLabel timerLabel;        // shows timer countdown
    private Timer gameTimer;          // counts down customer timer
    private final JLabel orderLabel;        // customer order
    private final JLabel selectionLabel;    // shows what player has
clicked
    private boolean jumpscareTriggered = false; // prevents double
jumpscare

    //----------------------- Background ----------------------------
    // background image
    private final Image bgImage = new ImageIcon("bg.png").getImage();

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.drawImage(bgImage, 0, 0, getWidth(), getHeight(), this);
    }
    //----------------------- Style Helper ----------------------------
    // reusable method to set font and color for any label
    private void styleLabel(JLabel label, int size) {
        label.setFont(new Font("Courier New", Font.BOLD, size));
        label.setForeground(Color.WHITE);
    }
    //----------------------- Game Panel ----------------------------
    public GamePannel(JFrame frame) {
        gameManager = new GameManager();
        gameManager.setCoin();   // start money
        gameManager.setRent();   // start rent
        recipe = new Recipe();
        playerSelection = new ArrayList<>();
        //----------------------- JLabel ----------------------------
        coinLabel = new JLabel("Coins: 500    ");
        dayLabel = new JLabel("Day: 1");
        timerLabel = new JLabel("Time: 30");
        customerLabel = new JLabel();
        selectionLabel = new JLabel("Selected: none"); // shows clicked
ingredient

        styleLabel(coinLabel, 18);
        styleLabel(dayLabel, 18);
        styleLabel(timerLabel, 18);
        selectionLabel.setHorizontalAlignment(SwingConstants.CENTER);
        styleLabel(selectionLabel, 24);
```

```
//----------------------- Top Panel 2 rows ----------------------------
// row 1: coins and day | row 2: timer on its own line
JPanel topPanel = new JPanel(new GridLayout(2, 1));
topPanel.setOpaque(false);
topPanel.setPreferredSize(new Dimension(800, 80));

JPanel row1 = new JPanel(); // coins and day row
row1.setOpaque(false);
row1.add(coinLabel);
row1.add(dayLabel);

JPanel row2 = new JPanel(); // timer row
row2.setOpaque(false);
row2.add(timerLabel);

topPanel.add(row1);
topPanel.add(row2);
add(topPanel, BorderLayout.NORTH);
//----------------------- JLabel Ingredients --------------------------
--
JButton riceBtn = new JButton("Rice");
riceBtn.setPreferredSize(new Dimension(100, 40));
riceBtn.addActionListener(e -> {
    playerSelection.add("Rice");
    updateSelection();
});
JButton ikuraBtn = new JButton("Ikura");
ikuraBtn.setPreferredSize(new Dimension(100, 40));
ikuraBtn.addActionListener(e -> {
    playerSelection.add("Ikura");
    updateSelection();
});
JButton crabBtn = new JButton("Crab");
crabBtn.setPreferredSize(new Dimension(100, 40));
crabBtn.addActionListener(e -> {
    playerSelection.add("Crab");
    updateSelection();
});
JButton shrimpBtn = new JButton("Shrimp");
shrimpBtn.setPreferredSize(new Dimension(100, 40));
shrimpBtn.addActionListener(e -> {
    playerSelection.add("Shrimp");
    updateSelection();
});
JButton tunaBtn = new JButton("Tuna");
tunaBtn.setPreferredSize(new Dimension(100, 40));
tunaBtn.addActionListener(e -> {
    playerSelection.add("Tuna");
    updateSelection();
});
JButton seaweedBtn = new JButton("Seaweed");
seaweedBtn.setPreferredSize(new Dimension(100, 40));
seaweedBtn.addActionListener(e -> {
    playerSelection.add("Seaweed");
    updateSelection();
});
```

```
JButton salmonBtn = new JButton("Salmon");
salmonBtn.setPreferredSize(new Dimension(100, 40));
salmonBtn.addActionListener(e -> {
    playerSelection.add("Salmon");
    updateSelection();
});
//---------------------- Other Buttons ----------------------------
JButton submitBtn = new JButton("Submit");
submitBtn.setPreferredSize(new Dimension(125, 30));
submitBtn.addActionListener(e -> checkOrder());

JButton recipeBtn = new JButton("Recipe Book");
recipeBtn.setPreferredSize(new Dimension(125, 30));
recipeBtn.addActionListener(e -> showRecipe());

JButton clearBtn = new JButton("Clear");
clearBtn.setPreferredSize(new Dimension(125, 30));
clearBtn.addActionListener(e -> {
    playerSelection.clear();
    selectionLabel.setText("Selected: none");
});
//----------------------- Bottom Panel ----------------------------
JPanel bottomPanel = new JPanel(new GridLayout(2, 1));
bottomPanel.setOpaque(false);
JPanel ingredientPanel = new JPanel(); // row 1
ingredientPanel.setOpaque(false);
ingredientPanel.add(riceBtn);
ingredientPanel.add(shrimpBtn);
ingredientPanel.add(salmonBtn);
ingredientPanel.add(tunaBtn);
ingredientPanel.add(crabBtn);
ingredientPanel.add(ikuraBtn);
ingredientPanel.add(seaweedBtn);

JPanel actionPanel = new JPanel(); // row 2
actionPanel.setOpaque(false);
actionPanel.add(recipeBtn);
actionPanel.add(clearBtn);
actionPanel.add(submitBtn);
bottomPanel.add(ingredientPanel); // ingredients on top
bottomPanel.add(actionPanel);     // submit and recipe below
add(bottomPanel, BorderLayout.SOUTH);

//----------------------- Center Panel ----------------------------
    JPanel centerPanel = new JPanel(new BorderLayout());
    centerPanel.setOpaque(false);
    orderLabel = new JLabel("Order: ", SwingConstants.CENTER);
    styleLabel(orderLabel, 22);
    centerPanel.add(orderLabel, BorderLayout.NORTH);     // order on top
    centerPanel.add(customerLabel, BorderLayout.CENTER); // customer in
middle
    centerPanel.add(selectionLabel, BorderLayout.SOUTH); // selection at
bottom
    add(centerPanel, BorderLayout.CENTER);
    SoundManager.playBGM(); // start music
    spawnCustomer();        // spawn first customer
}
```

```
//----------------------- Update Selection Label -----------------------
// shows what ingredients player has clicked so far
private void updateSelection() {
    selectionLabel.setText("Selected: " + String.join(" + ",
playerSelection));
}
//----------------------- Check Order ----------------------------
public void checkOrder() {
    ArrayList<String> required =
currentCustomer.getOrder().getIngredients();
    if (playerSelection.size() == required.size() &&
playerSelection.containsAll(required))
        // doesn't allow player to click same things twice and pass
    {
        if(gameManager.getDay()>3){
           gameManager.addCoin((gameManager.getDay()*12)-
gameManager.getDay()); //more day = more income per order
        } else {
             gameManager.addCoin(45); // correct = add coin
        }
        gameManager.addCustomer();
        updateLabels();
        if (gameManager.getCustomerCount() >= 5) { // 5 customer/days
            endDay();
        } else {
            spawnCustomer();
        }
    } else {
        if(gameManager.getDay() > 3) {
            currentCustomer.react(gameManager, 50); // harder days =
bigger penalty = mistake -> pay more
        } else {
            currentCustomer.react(gameManager); // normal penalty days 1
        }
        playerSelection.clear();              // reset after wrong order
        selectionLabel.setText("Selected: none"); // reset label
        if (currentCustomer instanceof Ghost ghost) { // check if
jumpscare should trigger
            if (ghost.isTriggerJumpscare()) {
                triggerJumpscare();
            }
        } else {
            // regular customer anger meter -> rage quits
            if (currentCustomer.getMistakeCount() >= 3) {
                gameManager.takeCoin(50); // extra penalty
                spawnCustomer();          // customer leaves angry
                JOptionPane.showMessageDialog(null,"Customer Angry!\n" +
"They just left..." ,
                        "Customer left",
                        JOptionPane.WARNING_MESSAGE);
            }
        }
    }
}
```

```
//----------------------- Spawn Customer -----------------------------
public void spawnCustomer() {
    jumpscareTriggered = false; // reset jumpscare flag for new customer
    if (gameManager.isGameOver()) { //check game over before spawn new
customer
        SoundManager.playGameOver();
        JOptionPane.showMessageDialog(null, "GAME OVER!");
        System.exit(0);
        return;
    }
    Ghost ghost = gameManager.spawnGhost();
      if (ghost != null) {
        currentCustomer = ghost; // spawn ghost if chance hits
        SoundManager.stopBGM(); // stop normal music
        SoundManager.playSus(); // play sus music for ghost!
    } else {
        currentCustomer = new Customer(); // spawn regular customer
        SoundManager.stopSus();  // stop sus music
        SoundManager.playBGM(); // make sure BGM playing for regular
customer
    }
    currentCustomer.setOrder(recipe.getRandomDish()); // set new random
order
    orderLabel.setText("Order: " +
currentCustomer.getOrder().getDishName());
    currentCustomer.setTimeLimit(25);

    // scale customer image to fit screen
    ImageIcon icon = new ImageIcon(currentCustomer.getCustomerPic());
    Image scaled = icon.getImage().getScaledInstance(450, 450,
Image.SCALE_SMOOTH);
    customerLabel.setIcon(new ImageIcon(scaled));
    playerSelection.clear();                  // clear old selection
    selectionLabel.setText("Selected: none"); // reset selection label
    startTimer();
}
//----------------------- Recipe Book -----------------------------
public void showRecipe() {
    ImageIcon recipeImage = new ImageIcon("Recipe.png");
    Image scaled = recipeImage.getImage().getScaledInstance(800, 500,
Image.SCALE_SMOOTH);
    JOptionPane.showMessageDialog(null,
            "",
            "Recipe Book",
            JOptionPane.PLAIN_MESSAGE,
            new ImageIcon(scaled));
}
//----------------------- Update Labels -----------------------------
public void updateLabels() {
    coinLabel.setText("Coins: " + gameManager.getCoin());
    dayLabel.setText("Day: " + gameManager.getDay());
    timerLabel.setText("Time: " + currentCustomer.getTimeRemaining());
    orderLabel.setText("Order: " +
currentCustomer.getOrder().getDishName());
}
```

```
//----------------------- Start Timer -----------------------------
public void startTimer() {
    if (gameTimer != null) gameTimer.stop(); // stop old timer
    gameTimer = new Timer(1000, e -> {
        currentCustomer.decrementTime();      // decrease time
        updateLabels();
        if (currentCustomer.getTimeRemaining() <= 0) { // if time run
out
            gameTimer.stop();
            currentCustomer.onLeave(gameManager); // customer leaves
            if (currentCustomer instanceof Ghost) {
                // ghost leave = jumpscare then game over
                if (!jumpscareTriggered) { // only trigger if not
already triggered
                    triggerJumpscare();
                }
            } else {
                spawnCustomer(); // regular customer leave
            }
        }

        if(Math.random() < 0.01) { // 1% chance every second
            SoundManager.playWhisper();
        }
        if(Math.random() < 0.01) { // 1% chance every second
            SoundManager.playIseeyou();
        }
    });
    gameTimer.start();
}

//----------------------- Jump Scare -----------------------------
private Timer heartTimer;
public void triggerJumpscare() {
    if (gameTimer != null) gameTimer.stop(); //stop time while jumpscare
to prevent when ghost timer run out player will get jumpscare again
    if (jumpscareTriggered) return; // prevent double jumpscare
    jumpscareTriggered = true;      // mark as trigger
    SoundManager.stopBGM();  // stop BGM
    SoundManager.stopSus();  // stop sus music
    SoundManager.playJump(); // play scream sound

    if (currentCustomer instanceof Ghost ghost) {
        // fullscreen on top
        JWindow scary = new JWindow();
        scary.setSize(800, 800);
        scary.getContentPane().setBackground(Color.BLACK); // blackbg

        ImageIcon gifIcon = new ImageIcon(ghost.getJumpscareImage());
        JLabel jumpLabel = new JLabel(gifIcon);
        jumpLabel.setBackground(Color.BLACK);
        jumpLabel.setOpaque(true);
        scary.add(jumpLabel);
        scary.setLocationRelativeTo(null); // center on screen
        scary.setVisible(true);            // show window
        scary.setAlwaysOnTop(true);
```

```
          heartTimer = new Timer(4500, e -> { // wait 6 seconds for
scream to finish
                scary.dispose();          // close scary window
                SoundManager.playHeart(); // play heartbeat after scare
                // check game over
                if (gameManager.isGameOver()) {
                    SoundManager.playGameOver();
                    JOptionPane.showMessageDialog(null, "GAME OVER!");
                    System.exit(0);
                }
                // ghost type 3 = force exit after scare
                if (ghost.getGhostType() == 3) {
                    JOptionPane.showMessageDialog(null,
                            "λoՈ HaΛE вEEИ MaʁN...",
                            "ᕽᖷᕮᒣ ᘔᕼᒣ",
                            JOptionPane.WARNING_MESSAGE);
                    System.exit(0);
                }
                spawnCustomer();
                heartTimer.stop();      // stop heart timer
                SoundManager.playBGM();// restart BGM after scare
            });
            heartTimer.start(); // start heart timer
        }
    }
//----------------------- End of the Day ---------------------------
    public void endDay() {
        gameTimer.stop();
        int servedToday = gameManager.getCustomerCount(); // save values
        double rentToday = gameManager.getRent();
        gameManager.nextDay(); // resets customerCount, rent
        // show end of day sum
        JOptionPane.showMessageDialog(null,
                "Day " + (gameManager.getDay() - 1) + " complete!\n" +
// -1
                        "Customers served: " + servedToday + "\n" +
                        "Rent paid: " + (gameManager.getRent() - 50) +
"\n" +
                        "Remaining Coins: " + gameManager.getCoin(),
                "End of Day",
                JOptionPane.INFORMATION_MESSAGE);
        if (gameManager.isGameOver()) { // check game over after pay
rent
            SoundManager.playGameOver();
            JOptionPane.showMessageDialog(null, "GAME OVER!");
            System.exit(0);
        } else {
            spawnCustomer(); // new day = new customer
        }
    }
}
```

8.1.3 Class GameManager :

```
public class GameManager {
    private double coin;
    private int day = 1;
    private double rent;
    private int mistake;
    private boolean gameOver;
    private double jumpScare;
    private int customerCount;
    private boolean ghostSpawnedToday;

    public int getDay(){ return day;}
    public int getMistake(){ return mistake;}
    public void setCoin() {
        this.coin = 500; //start money
    }
    public double getCoin(){return coin;}
    public void addCoin(double coin){ //add money if correct order
            this.coin += coin;
    } // add coin
    public void takeCoin(double coin) {
        this.coin -= coin;//take coin
        setGameOver(); //check game over
    }
    public void setRent(){
        this.rent = 250; //start rent
    }
    public double getRent() { return rent; }
    public void payRent(){this.coin -= this.rent;} //pay rent
    public void takeHalf() {this.coin /=2;}
    public boolean isGhostSpawnedToday() { return ghostSpawnedToday; }
// get ghost chance
    public void setGhostSpawnedToday(boolean value) {
this.ghostSpawnedToday = value; }
    public int getCustomerCount() { return customerCount; }
    public double getJumpScare() { return jumpScare; }
    //-----------------------Add Mistake-----------------------------
    public void addMistake() { //add mistake if order wrong(total
mistake reset everyday)
        mistake++;
        setGameOver(); // check if game over
    }
    //-----------------------Game Over-----------------------
    public void setGameOver(){ // money <= 0 gameover
        if(this.coin <= 0){
            this.gameOver = true;
        }
    }
    public void forceGameOver() {this.gameOver = true;}
    public boolean isGameOver(){return gameOver;}
```

```
//----------------------Add Customer--------------------------------
    public void addCustomer(){ // add customer for each day
        customerCount++;
    } // add customer the nextday from customer counter the day before
    public boolean shouldSpawnGhost() {
        if(!ghostSpawnedToday && Math.random() < jumpScare) {
            ghostSpawnedToday = true;
            return true;
        }
        return false;
    }
    //----------------------Spawn Ghost----------------------------
   public Ghost spawnGhost() {
        if(shouldSpawnGhost()) {
           int randomType = (int)(Math.random() * 5) + 1;
            return new Ghost(randomType, this);
        }
        return null; // no ghost today
    }
    //----------------------Next Day-----------------------------
    public void nextDay(){//at the end of the day
        ghostSpawnedToday = false;
        payRent();
        setGameOver(); // check if game over
        day++;
        rent += 50;
        jumpScare += 0.3;
        if(jumpScare > 0.8) jumpScare = 0.8;
        mistake = 0; // reset mistake
        customerCount = 0; //reset customer count
    }
}
```

8.1.4 Class GameChar :

```
public abstract class GameChar {
    public abstract void react(GameManager gm);
    //what each customer react when get wrong order
    public abstract void onLeave(GameManager gm);
    //what each customer do when timer run out
}
```

8.1.5 Class Customer :

```
public class Customer extends GameChar {
    private int timeLimit;
    private int timeRemaining;
    private int mistakeCount;
    private Menu order;
    private String customerPic;
```

```
//-------------------------Customer Pic--------------------------
    public Customer() {
        String[] customerImages = {
                "Customer/customer0.png",
                "Customer/customer1.png",
                "Customer/customer2.png",
                "Customer/customer3.png",
                "Customer/customer4.png",
                "Customer/customer5.png",
                "Customer/customer6.png",
                "Customer/customer7.png",
                "Customer/customer8.png",
                "Customer/customer9.png",
                "Customer/customer10.png",
                "Customer/customer11.png",
                "Customer/customer12.png"
        };
        int randomIndex = (int)(Math.random() * customerImages.length);
        this.customerPic = customerImages[randomIndex];
    }
    public void setCustomerPic(String pic) { this.customerPic = pic; }
    public int getTimeRemaining() { return timeRemaining; }
    public int getMistakeCount() { return mistakeCount; }
    public Menu getOrder() { return order; } // get order
    public String getCustomerPic() {return customerPic;}
    public void setOrder(Menu order) { this.order = order; }
    public void setTimeLimit(int timeLimit) {
        this.timeLimit = timeLimit;
        this.timeRemaining = timeLimit; // start countdown
    }
    //-----------------------Timer run down-------------------------
    public void decrementTime() {
        if(timeRemaining > 0) {
            timeRemaining--;
        }
    }
        public void addMistake() {
        mistakeCount++; // this specific customer's mistakes
    }
    //-------------------------Behavior for customer---------------------
    @Override
    public void react(GameManager gm){ //when wrong order
        addMistake();
        gm.addMistake();
        gm.takeCoin(35);
    };
    public void react(GameManager gm, int penalty) { // more penalty
        addMistake();
        gm.addMistake();
        gm.takeCoin(penalty);
    }
    @Override
    public void onLeave(GameManager gm2 ){ //when time run out
        gm2.takeCoin(45);
    };
}
```

8.1.6. Class Ghost :

```
public class Ghost extends Customer  {
    private int ghostType;
    private String jumpscareImage;
    private boolean triggerJumpscare;
    private GameManager gm;
    //-------------------Constructor-------------------
    public Ghost(int ghostType, GameManager gameManager) {
        super();
        this.gm = gameManager;
        this.ghostType = ghostType;

        String[] ghostImages = {
                "Ghost/ghost.png",
                "Ghost/ghost_face.png",
                "Ghost/ghost_forceexit.png",
                "Ghost/ghost_takehalf.png",
                "Ghost/foxy.png"
        };
        String[] jumpscareImages = {
                "Ghost/jump_ghost.png",
                "Ghost/ghost_face_kill.png",
                "Ghost/jump_force.png",
                "Ghost/jump_takehalf.png",
                "Ghost/jump_foxy.gif"
        };
        setCustomerPic(ghostImages[ghostType - 1]); //  sets parent's
customerPic as ghost one
        this.jumpscareImage = jumpscareImages[ghostType - 1]; // change
to jumpscare pic
    }
    //-------------------Getters-------------------
    public String getJumpscareImage() { return jumpscareImage; }
    public boolean isTriggerJumpscare() { return triggerJumpscare; }
    public int getGhostType() { return ghostType; }
    //-------------------React-------------------
    @Override
    public void react(GameManager gm) {
        switch(ghostType) {
            case 1:
                triggerJumpscare = true;
                break;
            case 2:
                triggerJumpscare = true;
                gm.forceGameOver();
                break;
            case 3:
                triggerJumpscare = true;
                break;
            case 4:
                triggerJumpscare = true;
                gm.takeHalf();
                break;
```

```
            case 5:
                triggerJumpscare = true;
                break;
        }
    }

      @Override
public void react(GameManager gm, int penalty) {
    // call the normal ghost reaction
    this.react(gm);
}
    //-------------------OnLeave-------------------
    @Override
    public void onLeave(GameManager gm) {
        gm.forceGameOver();
    }
}
```

8.1.7 Class Menu :

```
public class Menu {
    private final String dishName;
    private final ArrayList<String> ingredients;

    public Menu(String dishName, ArrayList<String> ingredients) {
        this.dishName = dishName;
        this.ingredients = ingredients;
    }

    public String getDishName() { return dishName; }
    public ArrayList<String> getIngredients() { return ingredients; }
}
```

8.1.8 Class Recipe :

```
public class Recipe {
    private ArrayList<Menu> allDishes; //all dish available
    public Recipe() {
        allDishes = new ArrayList<>(); // for add all dish
        setupRecipes(); // all ingredient we need for dish
    }

    private void setupRecipes() {
        // Salmon Roll
        ArrayList<String> salmonroll = new ArrayList<>();
        salmonroll.add("Rice");
        salmonroll.add("Salmon");
        salmonroll.add("Seaweed");
        allDishes.add(new Menu("Salmon Roll", salmonroll));

        //Salmon Maki
        ArrayList<String> salmonmaki = new ArrayList<>();
        salmonmaki.add("Rice");
        salmonmaki.add("Salmon");
        allDishes.add(new Menu("Salmon Maki", salmonmaki));
```

```
            // ikura maki
        ArrayList<String> ikuramaki = new ArrayList<>();
        ikuramaki.add("Rice");
        ikuramaki.add("Ikura");
        ikuramaki.add("Seaweed");
        allDishes.add(new Menu("Ikura Maki", ikuramaki));

        //Crab maki
        ArrayList<String> crabmaki = new ArrayList<>();
        crabmaki.add("Rice");
        crabmaki.add("Crab");
        crabmaki.add("Seaweed");
        allDishes.add(new Menu("Crab Maki", crabmaki));

        //Tuna Maki
        ArrayList<String> tunamaki = new ArrayList<>();
        tunamaki.add("Rice");
        tunamaki.add("Tuna");
        allDishes.add(new Menu("Tuna Maki", tunamaki));

        //Shrimp Maki
        ArrayList<String> shrimpmaki = new ArrayList<>();
        shrimpmaki.add("Rice");
        shrimpmaki.add("Shrimp");
        allDishes.add(new Menu("Shrimp Maki", shrimpmaki));
    }
    public ArrayList<Menu> getAllDishes() { return allDishes; }
    public Menu getRandomDish() { //random dish for each customer
        int randomIndex = (int)(Math.random() * allDishes.size());
        return allDishes.get(randomIndex);
    }
}
```

8.1.9 Class SoundManager :

```
public class SoundManager {
    private static Clip bgmClip; // stored so we can stop
    private static Clip susClip; // stored so we can stop

    public static void playBGM() { //play loop forever
        try {
            AudioInputStream audio = AudioSystem.getAudioInputStream(new
File("sound/bg_sound.wav"));
            Clip clip = AudioSystem.getClip();
            clip.open(audio);
            clip.loop(Clip.LOOP_CONTINUOUSLY);
            bgmClip = clip; // store it so it can be stop
        } catch (Exception e) {System.out.println("Error: " +
e.getMessage());}
    }

      public static void stopBGM() { //to stop bg sound
```

```
            if(bgmClip != null) bgmClip.stop();
    }
    public static void playHeart(){
        try {
            AudioInputStream audio = AudioSystem.getAudioInputStream(new
File("sound/heart_sound.wav"));
            Clip clip = AudioSystem.getClip();
            clip.open(audio);
            clip.start();
        } catch (Exception e) {System.out.println("Error: " + e.getMessage());}
    }
    public static void playJump(){
        try {
            AudioInputStream audio = AudioSystem.getAudioInputStream(new
File("sound/Scream_sound.wav"));
            Clip clip = AudioSystem.getClip();
            clip.open(audio);
            clip.start();
        } catch (Exception e) {System.out.println("Error: " + e.getMessage());}
    }
    public static void playGameOver(){
        try {
            AudioInputStream audio = AudioSystem.getAudioInputStream(new
File("sound/gameover.wav"));
            Clip clip = AudioSystem.getClip();
            clip.open(audio);
            clip.start();
        } catch (Exception e) {System.out.println("Error: " + e.getMessage());}
    }
    public static void playSus(){
        try {
            AudioInputStream audio = AudioSystem.getAudioInputStream(new
File("sound/ghost_show.wav"));
            Clip clip = AudioSystem.getClip();
            clip.open(audio);
            clip.start();
            susClip = clip; // store it so it can be stop
        } catch (Exception e) {System.out.println("Error: " + e.getMessage());}
    }
    public static void stopSus() { //to stop bg sound
        if(susClip != null) susClip.stop();
    }
    public static void playWhisper() {
        try {
            AudioInputStream audio = AudioSystem.getAudioInputStream(new
File("sound/whisper_sound.wav"));
            Clip clip = AudioSystem.getClip();
            clip.open(audio);
            clip.start();
        } catch (Exception e) {System.out.println("Error: " + e.getMessage());}
    }
    public static void playIseeyou () {
        try {
            AudioInputStream audio = AudioSystem.getAudioInputStream(new
File("sound/iseeyou_sound.wav"));
            Clip clip = AudioSystem.getClip();
            clip.open(audio);
            clip.start();
        } catch (Exception e) {System.out.println("Error: " + e.getMessage());}
    }
}
```

8.2 คู่มือการใช้งาน

การใช้งานเบื้องต้น :

- ตัวเกมจะสุ่มลูกค้าและเมนูอาหารที่ลูกค้าสั่ง โดยผู้เล่นจะต้องกดเลือกวัตถุดิบเพื่อทำเมนูนั้น ๆ
- ปุ่ม “Recipe book” ใช้สำหรับดูวัตถุดิบที่ต้องใช้ในแต่ละเมนู
- ปุ่ม “Clear” ใช้สำหรับยกเลิกการเลือกวัตถุดิบ เมื่อผู้เล่นเลือกวัตถุดิบผิด
- ปุ่ม “Submit” ใช้สำหรับเสิร์ฟเมนูให้ลูกค้า
    - ถ้าเลือกวัตถุดิบถูก จำนวนเงินจะเพิ่มขึ้น 45 บาท และลูกค้าคนถัดไปจะแสดงขึ้นมา
    - ถ้าเลือกวัตถุดิบผิดหรือซ้ำ จำนวนเงินจะถูกหัก โดยลูกค้าแต่ละคนสามารถเสิร์ฟผิดได้ 2 ครั้ง ก่อนที่ลูกค้าจะโกรธและออกจากร้าน

หลักการดำเนินเกม :

- ผู้เล่นจะมีจำนวนเงินอยู่ที่ 500 บาท
- ผู้เล่นจะต้องจ่ายค่าเช่าเริ่มต้นที่ 250 บาท และมีการเพิ่มขึ้น 50 บาททุก ๆ วัน
- หลังจากวันที่ 3 ผ่านไป ราคาอาหารแต่ละเมนูจะเพิ่มขึ้น
- ถ้าผู้เล่นทำอาหารผิดบ่อยจะถูกหักเงินครั้งละ 35 บาท แต่หลังจากวันที่ 3 จะเพิ่มการลงโทษเป็นถูกหักครั้งละ 50 บาท
- การสุ่มลูกค้าผีจะมีทั้งหมด 5 ประเภท ซึ่งแต่ละประเภทจะมีการ Jumpscare และพฤติกรรมที่แตกต่างกันออกไป
- เกมจบก็ต่อเมื่อผู้เล่นมีจำนวนเงินเป็นศูนย์หรือติดลบ

8.3 วิธีรันโปรแกรม

การรันโปรแกรมผ่าน Terminal

1. ติดตั้ง Java JDK 17 หรือสูงกว่า

2. ดาวโหลด folder โดยไฟล์จะต้องอยู่ครบเพื่อให้โปรแกรมทำงานได้อย่างถูกต้อง

3. ไปยัง Terminal และใส่ command line ดังนี้
   *javac -encoding UTF-8 *.java*
   (เนื่องจากโปรแกรมมีการใช้อักษรพิเศษจึงต้องมีการเพิ่ม -encoding UTF-8 เพื่อไม่ให้เกิดการ error)

4. จากนั้นสามารถรันโปรแกรมได้โดย
   java Main

8.4 UML Diagram

